Contents

Volume 06 — Web Vulns, Nmap & Wireshark (L51–L62) **1**
L51: Web Application Vulnerability Scanning 1
L52: SQL Injection — Authentication Bypass Part 1 2
L53: SQL Injection — Error Based Part 2 2
L54: SQL Injection — Error Based from Web Application Part 3 3
L55: SQLMAP 4
L56: Cross Site Scripting 4
L57: File Upload Vulnerability 5
L58: The NMAP Tool — A Relook Part I 5
L59: The NMAP Tool — A Relook Part II 6
L60: The NMAP Tool — A Relook Part III 7
L61: Network Analysis Using Wireshark 7
L62: Summarization of the Course 8

Volume 06 — Web Vulns, Nmap & Wireshark (L51–L62)

> Ethical Hacking · NPTEL 106105217 · Prof. Indranil Sengupta
> **notes-3** — concept exam notes (what things are / what they do). No lab command dumps.

Web scanning, SQLi, XSS, file upload, Nmap/Wireshark roles, course wrap-up.

L51: Web Application Vulnerability Scanning

What it is / what it does

- **OWASP Top 10:** injection, broken auth, sensitive data exposure, XXE, broken access control, misconfig, XSS, insecure deserialization, vulnerable components, insufficient logging.
- **Web app architecture:** browser → web server → application server → database.
- **Scanning types:** passive (spider/crawl), active (inject payloads), authenticated (with credentials).
- **False positives:** scanner reports vulnerability that doesn't exist — always verify manually.
- **Scope:** stay within authorized URLs/parameters defined in engagement rules.

Defenses (one-liners)

- WAF (Web Application Firewall) — ModSecurity, Cloudflare
- Regular automated scanning in CI/CD pipeline
- Input validation and output encoding
- Keep frameworks and libraries patched

Exam lines

- OWASP Top 10 lists most **critical web application risks**.

- Burp Suite acts as **intercepting proxy** between browser and server.
- Active scanning **sends attack payloads**; passive only observes traffic.
- Directory brute-forcing discovers **hidden pages and admin panels**.
- Always **verify scanner findings** manually before reporting.

L52: SQL Injection — Authentication Bypass Part 1

What it is / what it does

- **SQL Injection (SQLi):** insert malicious SQL into application query via unsanitized user input.
- **Root cause:** string concatenation of user input into SQL query.
- **Authentication bypass:** inject SQL that makes WHERE clause always true.
- **Injection points:** login forms, search boxes, URL parameters, HTTP headers, cookies.
- **Types:** in-band (results in response), blind (no direct output), out-of-band (DNS/HTTP exfiltration).

Defenses (one-liners)

- **Parameterized queries (prepared statements)** — never concatenate user input
- Input validation (whitelist allowed characters)
- Least-privilege database accounts (no DROP/UNION privileges)
- WAF with SQLi rules (secondary defense, not primary)

Exam lines

- SQLi root cause: **unsanitized user input concatenated** into SQL query.
- ’ OR ‘1’=’1 makes WHERE clause **always true** → bypasses authentication.
- – (double dash) is SQL **comment** — ignores rest of query.
- **Prepared statements** separate SQL structure from data (primary defense).
- Single quote ’ in input often triggers **SQL error** revealing vulnerability.

L53: SQL Injection — Error Based Part 2

What it is / what it does

- **Error-based SQLi:** exploit verbose SQL error messages to extract database information.
- **MySQL functions:** extractvalue(), updatexml(), floor() + rand() for error-based extraction.
- **Information gathering:** database name, table names, column names, data values.
- **UNION-based injection:** append UNION SELECT to extract data from other tables.
- **UNION requirements:** same number of columns, compatible data types.

Defenses (one-liners)

- Disable verbose error messages in production (custom error pages)
- Parameterized queries / ORM with parameter binding
- Database user with minimal privileges (no information_schema access)
- WAF blocking UNION, extractvalue, information_schema keywords

Exam lines

- Error-based SQLi exploits **verbose database error messages** for data extraction.
- UNION SELECT requires **matching column count** and compatible types.
- ORDER BY N– technique finds **number of columns** in query.
- information_schema.tables reveals **table names** in MySQL.
- extractvalue() forces **XPath error** containing injected data (MySQL).

L54: SQL Injection — Error Based from Web Application Part 3

What it is / what it does

- **Practical exploitation workflow** on DVWA, WebGoat, or custom vulnerable apps.
- **DVWA security levels:** Low (no filtering) → Medium (basic filter) → High (CSRF token) → Impossible (prepared statements).
- **Filter bypass techniques:**
- Case variation: UnIoN SeLeCt
- Comment injection: UN/**/ION SELECT
- URL encoding: %27 OR %271%27=%271
- Double encoding, hex encoding
- **Blind SQLi introduction:** boolean-based (AND 1=1 vs AND 1=2) and time-based (SLEEP(5)).

Defenses (one-liners)

- Prepared statements at all security levels (DVWA “Impossible” level)
- Input type validation (integer fields → cast to int)
- CSRF tokens prevent automated exploitation (not a SQLi fix alone)
- Comprehensive WAF rules + rate limiting

Exam lines

- Numeric injection: no quotes needed when input is **not string-quoted** in query.
- Filter bypass: **case variation, comment injection, encoding**.
- Blind SQLi: **no error messages** — infer from response differences or timing.
- SLEEP(5) = **time-based blind** SQLi (MySQL).
- CSRF token protects form submission but **does not prevent SQLi** if token is obtained.

L55: SQLMAP

What it is / what it does

- **sqlmap:** automated SQL injection and database takeover tool.
- **Detection engine:** tests all SQLi types (boolean, error, union, stacked, time-based).
- **DBMS support:** MySQL, PostgreSQL, MSSQL, Oracle, SQLite, and more.
- **Post-exploitation:** dump tables, read files, OS shell (via stacked queries/UDF).
- **Tamper scripts:** modify payloads to evade WAF/filtering.

Defenses (one-liners)

- Parameterized queries (makes sqlmap detection fail)
- WAF blocking sqlmap signatures (User-Agent, request patterns)
- Database least privilege (no FILE privilege, no xp_cmdshell)
- Monitor for sqlmap User-Agent and timing-based scan patterns

Exam lines

- –dbs enumerates databases; –dump extracts **table data**.
- –os-shell requires **stacked queries** and DBA-level privileges.
- Tamper scripts (–tamper) modify payloads to **evade WAF filters**.
- –batch runs with **default answers** (non-interactive mode).

L56: Cross Site Scripting

What it is / what it does

- **XSS:** inject malicious JavaScript into web page viewed by other users.
- **Types:**
- **Reflected:** payload in URL/form; executed immediately (non-persistent).
- **Stored:** payload saved in database; executed for every visitor (persistent).
- **DOM-based:** client-side JavaScript processes untrusted input without server round-trip.
- **Impact:** session hijacking (cookie theft), defacement, keylogging, phishing, redirect.
- **Cookie theft:**
  .

Defenses (one-liners)

- **Output encoding** (HTML entity encode user data before rendering)
- Content Security Policy (CSP): script-src ‘self’
- HttpOnly flag on session cookies (prevents JavaScript access)
- Input validation (secondary defense — encoding is primary)

Exam lines

- Stored XSS is **persistent** — affects all users viewing the page.
- Reflected XSS requires **victim to click malicious link**.
- DOM-based XSS: vulnerability in **client-side JavaScript** (no server involvement).
- HttpOnly cookie flag prevents **JavaScript cookie theft**.
- CSP script-src restricts **which scripts can execute** on the page.

L57: File Upload Vulnerability

What it is / what it does

- **Unrestricted file upload:** server accepts any file type → attacker uploads web shell.
- **Web shell:** PHP/ASP/JSP script providing remote command execution on server.
- **Bypass techniques:**
- Double extension: shell.php.jpg
- MIME type spoofing: Content-Type: with PHP content
- Null byte: shell.php%00.jpg (legacy)
- Case variation: shell.PhP
- Polyglot file: valid image header + embedded PHP code
- **Impact:** remote code execution (RCE), server compromise, data theft.

Defenses (one-liners)

- Whitelist allowed extensions (not blacklist)
- Validate file content (magic bytes), not just extension/MIME
- Store uploads outside webroot; serve via separate domain
- Rename uploaded files (remove user-controlled names)

Exam lines

- Web shell provides **remote command execution** on the server.
- Whitelist validation is **stronger than blacklist** for file types.
- MIME type can be **spoofed** — validate magic bytes/content.
- Store uploads **outside webroot** to prevent direct execution.
- Polyglot file is valid image **and** contains executable code.

L58: The NMAP Tool — A Relook Part I

What it is / what it does

- **Host discovery deep-dive:** ARP scan (local), ICMP echo/timestamp/netmask, TCP SYN/ACK ping, UDP ping.
- **ARP scan (-PR):** most reliable on local LAN; doesn't traverse routers.
- **ICMP types:** echo (type 8), timestamp (13), netmask (17) — -PE, -PP, -PM.
- **TCP ping:** SYN to port 80 (-PS80), ACK to port 443 (-PA443).

- **List scan (-sL):** DNS resolution only, no packets to target (stealthiest).

Defenses (one-liners)

- Block ICMP at perimeter (note: -Pn bypasses discovery anyway)
- Monitor ARP traffic for scanning patterns
- Honeypot IPs in unused address space to detect scans

Exam lines

- ARP scan (-PR) works only on **local Ethernet segment**.
- -Pn skips host discovery — treats **all hosts as up**.
- -sL performs **reverse DNS only** — no packets to targets.
- ICMP echo = -PE; TCP SYN ping = -PS.
- Multiple discovery methods increase reliability through **firewalls**.

L59: The NMAP Tool — A Relook Part II

What it is / what it does

- **Port scan techniques recap:**
- -sS SYN (half-open, stealthy, requires root)
- -sT Connect (full handshake, logged)
- -sU UDP (slow, ICMP port unreachable = closed)
- -sN NULL, -sF FIN, -sX Xmas (RFC 793 behavior)
- **Port specification:** -p 22,80,443, -p 1-1000, -p- (all 65535), –top-ports 100.
- **Speed templates:** -T0 (paranoid) to -T5 (insane); default T3.
- **Parallelism:** –min-parallelism, –max-parallelism, –min-rate.

Defenses (one-liners)

- Port knocking for sensitive services
- Firewall default-deny with allowlist
- IDS rate-based alerting on -T4/-T5 scans
- Reduce attack surface (close unused ports)

Exam lines

- -sS = SYN scan; **half-open**, doesn’t complete handshake.
- -p- scans **all 65535 ports** (TCP or UDP depending on scan type).
- -T0 to -T5 control **scan speed** (T0 = slowest/stealthiest).
- NULL/FIN/Xmas scans exploit **RFC 793** response behavior.
- –reason shows **why** nmap classified port as open/closed/filtered.

L60: The NMAP Tool — A Relook Part III

What it is / what it does

- **Service/version detection (-sV):** probe open ports, match against nmap-service-probes database.
- **OS detection (-O):** TCP/IP stack fingerprinting (TTL, window size, TCP options).
- **NSE (Nmap Scripting Engine):** –script=default,vuln,auth.
- **Output and reporting:** -oA basename (all formats), –stylesheet for HTML.
- **Advanced evasion:** –data-length, –ip-options, –source-port, –proxies.

Defenses (one-liners)

- Keep services updated (version detection reveals patch level)
- OS fingerprint obfuscation (some OS allow TTL/window customization)
- NSE script detection at IPS/WAF layer
- Minimize exposed service information (remove banners)

Exam lines

- -sV probes open ports for **service name and version**.
- -O requires **at least one open and one closed port** for OS detection.
- -A = aggressive: OS + version + scripts + traceroute + **not stealthy**.
- -oA saves in **all formats** (normal, XML, grepable).
- NSE –script vuln runs **vulnerability detection** scripts.

L61: Network Analysis Using Wireshark

What it is / what it does

- **Wireshark:** GUI packet analyzer; captures and dissects traffic at frame level.
- **Capture filters (BPF):** applied during capture — host 192.168.1.1, port 80, tcp port 443.
- **Display filters:** applied post-capture — http, tcp.flags.syn==1, ip.addr==x.
- **Follow TCP/UDP stream:** reassemble conversation for readable analysis.
- **Use cases:** troubleshoot connectivity, detect malware C2, analyze attacks, forensic investigation.

Defenses (one-liners)

- Encrypt all traffic (TLS) — Wireshark sees only metadata without keys
- Network monitoring with automated anomaly detection
- Full packet capture (FPC) appliances for forensic retention
- DNS logging and Sinkholing for C2 detection

Exam lines

- Capture filter (BPF) applied **during** capture; display filter **after**.

- tcp.flags.syn==1 filters **SYN packets** (scan detection).
- Follow TCP Stream **reassembles** entire conversation for reading.
- Wireshark cannot decrypt TLS without **session keys** (SSLKEYLOGFILE).
- tshark is Wireshark's **command-line** counterpart.

L62: Summarization of the Course

What it is / what it does

- **Course arc recap:**

Exam lines

- Pentest phases: **recon → scan → exploit → maintain → report**.
- Defense-in-depth: multiple **independent security layers**.
- CIA triad + authentication + non-repudiation = core security goals.
- Always obtain **written authorization** before any security testing.
- Keep tools updated: Nmap NSE, Nessus plugins, Metasploit modules, sqlmap tamper scripts.